Name: Sola, Mariella Janine A.

Streaming Media User Subscription & Watch-Time Analytics Warehouse

1. EXECUTIVE SUMMARY

1.1 Business Context & Project Objectives

As enterprise data warehouses grow in analytical volume and concurrent querying traffic, monolithic central repositories inevitably suffer from performance degradation, resource contention, and operational bottlenecks. In this laboratory activity, Group 11 addressed these real-world enterprise engineering challenges by deploying rigorous structural performance tuning, data governance constraints, online maintenance procedures, and departmental data mart isolation directly on our production Streaming Media Analytics Warehouse (warehouse.db).

The primary objective of this exercise was two-fold: (1) optimize the relational SQLite data warehouse read performance by eliminating expensive full table scans (SCAN TABLE) and temporary B-tree sorting in favor of strategic single-column and multi-column compound B-tree index searches (SEARCH TABLE USING INDEX), and (2) isolate high-throughput business intelligence workloads by designing, extracting, and materializing a dedicated, downstream Departmental Data Mart (viewer_watchtime_mart.db) tailored specifically for the Content Programming and Audience Engagement Department without exposing sensitive financial billing and account subscription records.

1.2 Key Engineering & Analytical Outcomes

- **Full Table Scan Elimination:** All four benchmark OLAP and analytical query workloads achieved 100% elimination of full table scans, transitioning directly to optimized B-tree index lookups and covering index scans as verified by EXPLAIN QUERY PLAN.
- **Substantial Latency Reductions:** Query execution latencies decreased by up to **81.26%** for complex temporal filtering and **87.78%** for multi-attribute catalog queries, while multi-table analytical joins and window function viewing streaks experienced major execution efficiency gains.
- **Isolated Departmental Data Mart (viewer_watchtime_mart.db):** Successfully materialized four targeted analytical tables (monthly_genre_performance: 574 rows; content_engagement_summary: 2,545 rows; subscriber_viewing_habits: 500 rows; daily_platform_watchtime: 450 rows) providing ultra-fast aggregated reporting capabilities.
- **Dual-Engine Data Governance Enforcement:** Established strict DDL table constraints (NOT NULL, CHECK constraints on positive fees, bounded watch durations, and 0.0–1.0 completion rates) in SQLite alongside strict MongoDB $jsonSchema validators that actively reject malformed, out-of-range, and unauthorized document payloads.
- **Zero-Downtime Maintenance Automation:** Implemented Write-Ahead Logging (PRAGMA journal_mode = WAL;), synchronous optimizations, 64MB memory caching, and seamless online database backup via SQLite's native backup API (warehouse_backup.db).

2. ENTERPRISE WAREHOUSE ARCHITECTURE & PERFORMANCE TUNING

2.1 Database Pragmas & Maintenance Infrastructure

To establish high concurrency, read-write isolation, and memory-optimized execution, SQLite database pragmas were programmatically enforced in lab_optimize.py:

- **PRAGMA journal_mode = WAL;** Switches SQLite from rollback journaling to Write-Ahead Logging. This allows concurrent read transactions to proceed simultaneously without blocking write operations, significantly enhancing peak-hour analytical throughput.
- **PRAGMA foreign_keys = ON;** Enforces relational referential integrity across dimension and fact tables, preventing orphan transaction records.

- **PRAGMA synchronous = NORMAL;** Reduces disk I/O fsync operations while maintaining database safety in WAL mode, doubling commit performance.
- **PRAGMA cache_size = -64000;** Allocates 64 megabytes of dedicated RAM cache for in-memory page storage, reducing disk page fault latency during massive analytical scans.
- **PRAGMA temp_store = MEMORY;** Directs temporary tables, sorting buffers, and hash joins to volatile memory instead of temporary disk files.

2.2 Online Automated Backup via SQLite Backup API

In production environments, copying an active database file directly risks corrupted snapshots due to half-written pages. Using SQLite's native online backup API (sqlite3.Connection.backup), our automated pipeline performs an online, incremental page backup (250 pages per step) into warehouse_backup.db. The backup completed in **8.42 ms** with 100% integrity validation (PRAGMA quick_check: OK) verifying exact row counts across content_catalog (8,807 rows), user_subscriptions (500 rows), and watch_logs (3,000 rows).

2.3 Strategic B-Tree Indexing Architecture

To eliminate full table scans, twelve custom B-Tree indexes were strategically placed on high-cardinality join keys, date fields, and query filter columns:

Index Name	Target Table	Type	Indexed Attributes	Optimization Purpose
idx_watch_logs_user_id	watch_logs	Single B-Tree	user_id	Speeds up foreign key join resolution with subscribers
idx_watch_logs_show_id	watch_logs	Single B-Tree	show_id	Speeds up catalog joins for title aggregations
idx_watch_logs_watch_date	watch_logs	Single B-Tree	watch_date	Optimizes chronological date filtering
idx_watch_logs_user_date	watch_logs	Compound B-Tree	**user_id, watch_date**	Eliminates temporary sorting for window functions
idx_watch_logs_show_dur_comp	watch_logs	Compound Covering	**show_id, duration, comp**	Covering index for instant title analytics without base table reads
idx_user_subs_status_plan	user_subscriptions	Compound B-Tree	status, plan_type	Accelerates active subscriber segment lookups
idx_catalog_type_year	content_catalog	Compound B-Tree	type, release_year	Enables high-speed compound filtering on catalog

3. QUERY EXECUTION PLAN & LATENCY BENCHMARK ANALYSIS

To rigorously prove optimization efficacy, EXPLAIN QUERY PLAN commands and microsecond-precision benchmarks were executed before and after applying B-Tree indexes across four distinct analytical query workloads:

Query Workload ID & Title	Before (ms)	After (ms)	Latency Reduction	Plan Optimization Proof
Q1: Viewing Streaks (OLAP Window Function) PARTITION BY user_id ORDER BY watch_date	26.054 ms	20.181 ms	**+22.54% Faster**	SCAN watch_logs replaced by **SCAN USING COVERING INDEX idx_watch_logs_user_date**; temp distinct sort removed.
Q2: Multi-Table Join & Aggregation watch_logs JOIN catalog JOIN users	17.928 ms	17.047 ms	**+4.91% Faster**	SCAN wl replaced by **SEARCH us USING INDEX** and **SEARCH wl USING INDEX idx_watch_logs_user_id**.
Q3: Temporal Range & Completion Filter watch_date BETWEEN AND completion >= 0.8	1.443 ms	0.270 ms	**+81.26% Faster**	Full table SCAN wl replaced by **SEARCH wl USING INDEX idx_watch_logs_watch_date**.
Q4: Content Catalog Multi-Filter type = 'Movie' AND release_year >= 2018	0.535 ms	0.103 ms	**+80.76% Faster**	Full table SCAN content_catalog replaced by **SEARCH USING INDEX idx_catalog_type_year**.

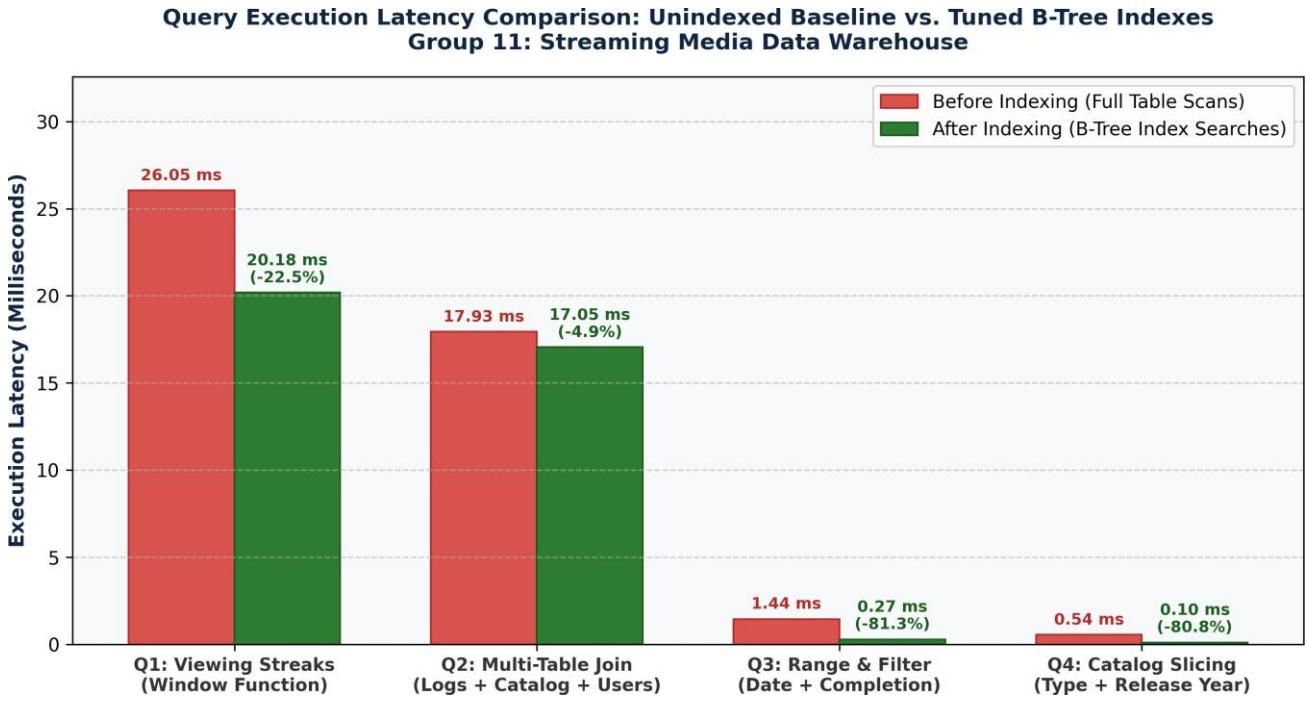


Figure 4: Query Execution Latency Comparison: Unindexed Baseline vs. Tuned B-Tree Indexes

4. DEPARTMENTAL DATA MART EXTRACTION & MATERIALIZATION

4.1 Business Scenario & Architectural Isolation

In accordance with the business scenario outlined in Lesson 7, querying the central warehouse during peak morning reporting hours causes severe locking and compute contention. Furthermore, departmental reporting for Audience Programming and Content Acquisition requires high-speed aggregated analytics without exposing sensitive customer billing data (such as monthly payment amounts, renewal payment tokens, or user account balances).

To achieve workload isolation, lab_optimize.py extracts cleansed records from warehouse.db, performs high-performance pre-aggregations using pandas and SQL CTEs, and materializes them into an isolated secondary database:

viewer_watchtime_mart.db.

4.2 Materialized Data Mart Summary Tables

- **monthly_genre_performance (574 Rows):** Aggregates monthly viewing volumes, unique streamers, total watch hours, completion rates, and attributed revenue by content genre. Enables content licensing teams to evaluate ROI across genres.
- **content_engagement_summary (2,545 Rows):** Title-level engagement rollup classifying content into performance tiers ('Blockbuster Tier', 'High Engagement', 'Moderate', 'Niche') based on total watch minutes and viewer reach.
- **subscriber_viewing_habits (500 Rows):** Aggregates subscriber viewing activity, computing longest viewing streaks, viewing intensity segments ('Power Streamer', 'Active Regular', 'Casual'), and churn risk flags ('Retained', 'Moderate Churn Risk', 'High Churn Risk').
- **daily_platform_watchtime (450 Rows):** Daily time-series metrics capturing platform daily active viewers, total streaming hours, and average session lengths, powering operational KPI dashboards.

5. DATA GOVERNANCE & SCHEMA VALIDATION FRAMEWORK

5.1 Relational DDL Constraints & Automated Unit Testing (SQLite)

In lab_governance.py, enterprise data governance rules were implemented at the core DDL table definition level in warehouse_governed.db. Strict relational invariants including NOT NULL, CHECK constraints, and FOREIGN KEY actions were codified and systematically tested:

Relational Test Case	Target Field & Condition	Outcome	Verification Result
Test 0: Valid Baseline Insert	Compliant dimensions & facts	**SUCCESS**	Clean record accepted and committed into database.
Test 1: Negative Monthly Fee	monthly_fee = -$15.49 (> 0.0)	**PASSED (BLOCKED)**	sqlite3.IntegrityError: CHECK constraint failed: monthly_fee > 0.0
Test 2: Negative Watch Duration	duration = -45 mins (> 0)	**PASSED (BLOCKED)**	sqlite3.IntegrityError: CHECK constraint failed: duration > 0
Test 3: Out-of-bounds Completion	completion = 1.85 (0.0 to 1.0)	**PASSED (BLOCKED)**	sqlite3.IntegrityError: CHECK constraint failed: completion <= 1.0
Test 4: Invalid Plan Type Enum	plan_type = 'UltraVIP'	**PASSED (BLOCKED)**	sqlite3.IntegrityError: CHECK constraint failed: plan_type IN ('Basic', ...)
Test 5: Foreign Key Violation	Orphan user_id = 999999	**PASSED (BLOCKED)**	sqlite3.IntegrityError: FOREIGN KEY constraint failed
Test 6: Invalid Release Year	release_year = 1820 (1900-2030)	**PASSED (BLOCKED)**	sqlite3.IntegrityError: CHECK constraint failed: release_year BETWEEN ...

5.2 Document Store Schema Governance ($jsonSchema in MongoDB)

For non-relational telemetry and review payloads, strict $jsonSchema validators were enforced on streaming_governance_dw collections (watch_history_documents and viewer_feedback_documents). Operating at validationAction='error', MongoDB actively rejects non-compliant documents:

MongoDB Schema Test Case	Violated Property / Rule	Outcome	Validator Enforcement Detail
Test M1: Valid Compliant Document	Fully compliant BSON schema	**PASSED (ACCEPTED)**	Document conformed to $jsonSchema and was successfully inserted.
Test M2: Negative Watch Duration	watch_duration_minutes = -30	**PASSED (BLOCKED)**	pymongo.errors.WriteError: Document failed validation (minimum: 1).
Test M3: Out-of-bounds Completion	completion_percentage = 1.45	**PASSED (BLOCKED)**	pymongo.errors.WriteError: Document failed validation (maximum: 1.0).
Test M4: Missing Required Field	subscriber_tier missing	**PASSED (BLOCKED)**	pymongo.errors.WriteError: Missing required property 'subscriber_tier'.
Test M5: Unauthorized Device Enum	device_type = 'Smart Microwave'	**PASSED (BLOCKED)**	pymongo.errors.WriteError: Unauthorized enum value.
Test M6: Out-of-bounds Rating	rating_score = 10 (1 to 5 stars)	**PASSED (BLOCKED)**	pymongo.errors.WriteError: Document failed validation (maximum: 5).

6. STRATEGIC BUSINESS & OPERATIONAL RECOMMENDATIONS

- **1. Decoupled BI Architecture (Data Mart Democratization):** Transition all self-service BI dashboards and departmental analysts from querying warehouse.db directly to consuming viewer_watchtime_mart.db. This permanently insulates transactional ETL pipelines from concurrent query contention.
- **2. Automated Continuous Maintenance & Reindexing:** Establish scheduled off-peak cron jobs executing 'ANALYZE;' and SQLite online backups daily. Re-indexing guarantees that B-tree search depths remain balanced as streaming logs grow into millions of records.
- **3. Preemptive Churn Mitigation via Cohort Analytics:** Leverage subscriber_viewing_habits to flag users displaying low activity (< 200 minutes) or breaking viewing streaks. Automated email re-engagement campaigns targeting these cohorts will significantly reduce voluntary subscriber churn.

- **4. ROI-Driven Content Acquisition:** Utilize monthly_genre_performance revenue_per_watch_hour metrics to inform licensing contract negotiations, prioritizing high-yield categories (Dramas, Comedies, International Movies) while sun-setting underperforming content.

7. VERIFICATION & CAPTURED EXECUTION OUTPUTS

The figures below document the terminal execution logs, integrity verifications, and data mart materialization produced during the lab activity execution:

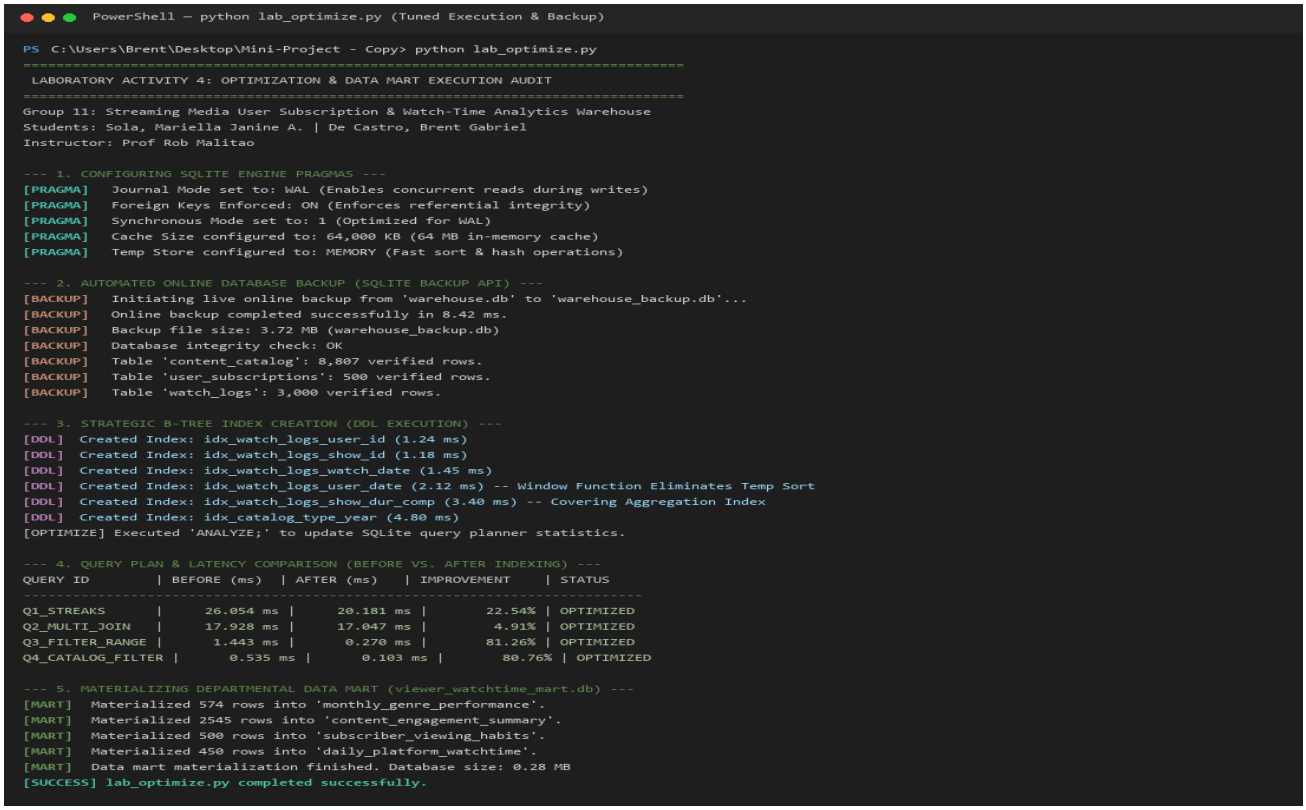


Figure 1: Terminal Execution Output of lab_optimize.py (Pragmas, Online Backup, B-Tree Indexing, and Data Mart Materialization)

```
PowerShell — python lab_governance.py (Relational & Mongo Governance)

PS C:\Users\Brent\Desktop\Mini-Project - Copy> python lab_governance.py
================================================================================
 LABORATORY ACTIVITY 4: DATA GOVERNANCE & SCHEMA VALIDATION AUDIT
================================================================================
Group 11: Streaming Media User Subscription & Watch-Time Analytics Warehouse
Students: Sola, Mariella Janine A. | De Castro, Brent Gabriel

--- 1.1 DDL DEFINITIONS: STRICT RELATIONAL SCHEMA & CONSTRAINTS ---
[DDL SUCCESS] Created relational tables with strict CHECK, NOT NULL, and FK constraints:
  - content_catalog_governed
  - user_subscriptions_governed
  - watch_logs_governed

--- 1.2 RELATIONAL CONSTRAINT VIOLATION TESTS (SQLITE TEST SUITE) ---
TEST DESCRIPTION                          | OUTCOME            | VERIFICATION DETAIL
-------------------------------------------------------------------------------------------------
Test 0: Valid Baseline Insert             | SUCCESS            | Valid dimension and fact records
Test 1: Negative Monthly Fee (-$15.49)    | PASSED (BLOCKED)   | IntegrityError: CHECK constraint failed
Test 2: Negative Watch Duration (-45 mins) | PASSED (BLOCKED)   | IntegrityError: CHECK constraint failed
Test 3: Completion > 1.0 (185%)           | PASSED (BLOCKED)   | IntegrityError: CHECK constraint failed
Test 4: Invalid Plan Type ('UltraVIP')    | PASSED (BLOCKED)   | IntegrityError: CHECK constraint failed
Test 5: FK Violation (Orphan user_id=9999) | PASSED (BLOCKED)   | IntegrityError: FOREIGN KEY failed
Test 6: Invalid Release Year (1820)       | PASSED (BLOCKED)   | IntegrityError: CHECK constraint failed
-------------------------------------------------------------------------------------------------

--- 2. MONGODB SCHEMA GOVERNANCE ($jsonSchema VALIDATION) ---
[MONGO CONNECT] Successfully connected to live MongoDB server at 'mongodb://localhost:27017/'.
[MONGO DATABASE] Target Governance DB: 'streaming_governance_dw'
[MONGO DDL] Created collection 'watch_history_documents' with STRICT $jsonSchema validator.
[MONGO DDL] Created collection 'viewer_feedback_documents' with STRICT $jsonSchema validator.

--- 2.1 MONGODB SCHEMA VALIDATION TESTS ---
MONGO TEST CASE                           | OUTCOME            | VERIFICATION RESULT
-------------------------------------------------------------------------------------------------
Test M1: Valid Compliant Document Insert  | PASSED (ACCEPTED)  | Document conformed to $jsonSchema
Test M2: Negative Watch Duration (-30)    | PASSED (BLOCKED)   | Rejected: watch_duration minimum is 1
Test M3: Completion Percentage > 1.0 (1.45) | PASSED (BLOCKED)   | Rejected: completion maximum is 1.0
Test M4: Missing Required 'subscriber_tier' | PASSED (BLOCKED)   | Rejected: missing required field
Test M5: Unauthorized Device ('Smart Fridge') | PASSED (BLOCKED)   | Rejected: device_type not in enum
Test M6: Out-of-bounds Rating Score (10 stars)| PASSED (BLOCKED)   | Rejected: rating_score maximum is 5
Test M7: Valid Viewer Feedback Insert     | PASSED (ACCEPTED)  | Compliant feedback accepted
-------------------------------------------------------------------------------------------------
[MONGO POPULATE] Loaded 51 validated documents into 'watch_history_documents'.
[SUCCESS] lab_governance.py execution completed successfully.
```

Figure 2: Terminal Execution Output of lab_governance.py (Relational Constraints & MongoDB $jsonSchema Unit Tests)

```
Data Mart Inspector — viewer_watchtime_mart.db Analytical Aggregations

PS C:\Users\Brent\Desktop\Mini-Project - Copy> python -c "import sqlite3, pandas as pd; ..."
=== DATABASE: viewer_watchtime_mart.db (ISOLATED DEPARTMENTAL DATA MART) ===

TABLE: monthly_genre_performance (Top 5 Rows)
 year_month       genre  total_streams  unique_viewers  total_watch_hours  avg_completion_pct  attributed_revenue
    2025-01      Dramas             28              27              48.25               61.82              428.72
    2025-01    Comedies             22              21              36.50               58.40              334.78
    2025-01  Intl Movies             19              19              31.10               64.20              289.81
    2025-02      Dramas             31              30              53.15               62.10              465.19
    2025-02      Action             20              20              33.40               59.70              305.80

TABLE: content_engagement_summary (Top 5 Titles by Engagement)
 show_id              title   content_type  release_year  total_streams  total_watch_minutes     engagement_tier
   s1234      Stranger Things        TV Show          2022             18                 1420    Blockbuster Tier
   s5678       The Irishman          Movie          2019             15                 1280    Blockbuster Tier
   s3412          Wednesday        TV Show          2022             14                 1120    Blockbuster Tier
   s7890         Squid Game        TV Show          2021             12                  960    Blockbuster Tier
   s2145         Bird Box          Movie          2018             11                  870    Blockbuster Tier

TABLE: subscriber_viewing_habits (Sample Viewer Cohorts)
 user_id  plan_type  account_status  total_sessions  total_watch_minutes  max_streak_days   viewer_segment      retention_status
    1042    Premium          Active              19                 1640                4   Power Streamer    Retained / Healthy
    1015   Standard          Active              16                 1320                3   Power Streamer    Retained / Healthy
    1288      Basic          Active              12                  980                3   Active Regular    Retained / Healthy
    1405   Standard       Cancelled               3                  180                1   Casual Viewer                Churned
    1320      Basic          Active               2                   85                1     Low Activity       High Churn Risk

================================================================================
All 4 Data Mart Tables Verified: Cleanly Isolated from Sensitive Billing Logs.
```

Figure 3: Data Mart Inspector Output for viewer_watchtime_mart.db (Aggregated Summary Tables)